

DIV-TBAT algorithm for test suite reduction in software testing

ISSN 1751-8806

Received on 1st July 2017

Revised 25th January 2018

Accepted on 9th March 2018

E-First on 2nd May 2018

doi: 10.1049/iet-sen.2017.0130

www.ietdl.org

Shounak Rushikesh Sugave¹ ✉, Suhas Haribhau Patil², B. Eswara Reddy³

¹JNTUA, Ananthapuramu, Andhra Pradesh, India

²Bharati Vidyapeeth University College of Engineering, Pune, Maharashtra, India

³JNTUA College of Engineering, Kalikiri, Chittoor District, Andhra Pradesh, India

✉ E-mail: shounaksugave16@gmail.com

Abstract: Researchers have investigated different approaches to maintain the minimum cost and effort in regression testing. Here, test suite reduction is a common technique to decrease the cost of regression testing by removing the redundant test cases from the test suite and then, obtaining a representative set of test cases that still yield a high level of code coverage. Accordingly, here, the authors have developed two various techniques for test suite reduction. In the first technique, ATAP measure is newly developed to find the reduced test suite with the help of greedy search algorithm. In the second technique, DIV-TBAT (Diversity-based BAT) algorithm is newly devised based on the mechanisms of Boolean logic within BAT algorithm which improve diversity during the search process. The proposed techniques are experimented using eight programs from SIR subject programs and the performance study is conducted using nine different evaluation metrics based on different research questions. The comparative analysis is performed with the existing algorithms like GreedyRatio, GreedyElrreplaceability, diversity-based genetic algorithm, TBAT, and TAP, to prove the performance improvement over the eight software programs considered.

1 Introduction

The adaptation of the test suite minimisation technique filters the test suite in the program, so that the filtered test suite can detect the faults as much as possible using the handy resources available [1]. The primary intention to create the test suite is to expose the faults, thereby to attain an effective fault detection method. To attain the measurement with the consistent results, different types of the code coverage are used by the test suite as an alternative for the fault detection capability valuation. During the test planning, the test coverage is chosen for the test suite. Test coverage is available in different forms like requirement coverage, interfaces coverage, design coverage, and code coverage [2].

The test suite minimisation also comprises some of the problems during the minimisation process. The minimisation problem can be streamlined as a minimum set cover problem [3]. The minimum set cover problem has been shown to be non-deterministic problem complete. Many of the techniques for the test suite reduction have been proposed in [4–19]. A test suit minimisation technique is said to be optimal only if it retains the most effective test cases which are capable of satisfying the number of the test requirements in addition to effectively detect the faults in existence. Another important aspect accompanied with the reduction technique is the usage of the test coverage. Some of the coverage criteria are branch coverage, statement coverage, data flow coverage, MC/DC coverage, call stack coverage [20] etc.

In this paper, we have developed a new measure called ATAP by modifying the TAP measure for test suite reduction. Summarising, the contributions of this paper are:

- New measure:* A newly proposed measure called ATAP measure (Adaptive Test cases which is already included in Pool-based measure) which is the extension of the TAP measure given in [21].
- New objective functions:* We have designed a new objective function for evaluating the test suite reduction. The generation of reduced test suite should be based on the two constraints like (a) satisfying all test requirements, (b) minimising the cost value.

- New algorithm:* We have presented a new algorithm called DIV-TBAT (Diversity-based BAT), which is the modified TBAT algorithm [22] by inserting the diversity into the algorithm procedure. The injection of diversity is done using the Boolean logic which preserves the diversity in the position of the bats.
- Validation:* To validate the seven algorithms, nine evaluation metrics and eight software programs are used. Also, we have formulated nine research questions based on nine metrics to show the performance improvement of the proposed methods.

2 Problem statement

Let us assume that the input of the proposed test suite reduction approach is the test suite which is represented as $T = \{t_{ij}; 0 < i < n; 0 < j < m\}$. Here, n is the number of test case available to test the software program and m is the number of test requirement. Test requirement means that the branch coverage of the test case. Based on this representation of the test suite, every value, t_{ij} , within the test suite T may have either 0 or 1. Here, 0 indicates that i th test case can cover the j th branch in the software program. This can be further interpreted as the i th test case can satisfy the j th test requirement. Every test cases are applied to the software programs and the execution time is computed. The execution time is assumed to be the cost of the test case. Here, cost is denoted as, $\text{Cost} = \{g_i; 0 < i < n\}$, where g_i is the cost of the i th test case.

The problem formulated based on this assumption is to reduce the size of the test suite from n test cases to $n - x$ test cases by satisfying all the test requirements. Accordingly, the reduced test suite can be represented as like, $T_R = \{t_{k_j}; 0 < i < n - x; k_1 < k_2 < \dots < k_{n-x}; 0 < j < m\}$. The reduction in test suite should be carried out by rightly identification of x test cases to be removed by satisfying the following constraint and the optimisation objective

```

1  Algorithm: GreedyATAP
2  Input:  $T \rightarrow$  Test suite
3            $Cost \rightarrow$  Cost vector
4  Output:
5            $ST \rightarrow$  Selected test cases
6  Begin
7            $ST = \{ \}$ ;
8  fill the entire test requirements in  $TR$ 
9           While ( $TR \neq \text{NULL}$ )
10            {
11             for each  $t$ 
12              $MC1=0$ ;  $C1=0$ ;
13             for each  $c$ 
14             if ( $t$  does not cover the requirement  $c$ )
15             {
16               $ATAP(t)=0$ ;
17             }
18             Elseif ( $c$  is only covered by  $t$ )
19             {
20               $ATAP(t) = \infty$ ;
21             }
22             Else
23             {
24              Find
25               $T_a, T_s$  and  $F_d$ 
26              Find  $C$  and  $MC$  using  $T_a, T_s$  and  $F_d$ 
27               $MC1 = MC1 + MC$ ;
28               $C1 = C1 + C$ ;
29              Endfor
30               $ATAP(t) = (MC1 - C1) / g(t)$ 
31            }
32            Endfor
33            Select  $tMAX$  which is a test case having maximum  $ATAP(t)$ 
34            Get  $c_c$  which is requirements by the selected  $tMAX$ 
35            Add  $tMAX$  to  $ST$ 
36            Remove  $c_c$  from  $TR$ 
37            }
38 Endwhile
39 End

```

Fig. 1 GreedyATAP search algorithm for test case reduction

$$(i) \quad K_j = \sum_{i=1}^{n-x} t_{ij} > 0; \quad \forall j \quad (1)$$

$$(ii) \quad \text{Min} \left(\sum_{i=1}^{n-x} g_i \right) \quad (2)$$

From the above equation, we identify that the summation of the j th column should be >0 . This indicates that the j th test requirement is satisfied if K_j is >0 . Also, the second constraints show the summation of cost value of every test cases should be minimum as much as possible to get the improved performance.

3 ATAP: a new measure for test suite reduction

This paper presents a new measure for evaluating test cases based on the three parameters like contribution, moving contribution, and adaptive decrement factor. These three parameters are considered three factors like number of test cases that can satisfy availed requirement, number of test cases that already satisfied requirement, and number of requirements satisfied by the corresponding test case.

3.1 Definition of ATAP measure

The evaluation of test cases is found out using the proposed formulae given below. In these formulae, we included three parameters like contribution, moving contribution, and cost. This provides the infinity to the test case when the test case t can only satisfy requirement c_s . Otherwise, the cumulative sum of

contribution of all the requirements will be subtracted from cumulative sum of the moving contribution of the requirements. Here, k is the number of test requirement availed and l is the number of test requirement already satisfied (see (3)) where $g(t)$ is the cost of the test case. The contribution and moving contribution of the above equation is defined as follows:

$$C(t, c_s) = \begin{cases} 0; & \text{if } t \text{ cannot satisfy } c_s \\ \left(\frac{1}{T_a} \right); & \text{if } t \text{ satisfies } c_s \end{cases} \quad (4)$$

$$MC(t, c_s) = \begin{cases} 0; & \text{if } t \text{ cannot satisfy } c_s \\ \left(\frac{1}{F_d(t) * T_s} \right); & \text{if } t \text{ satisfies } c_s \end{cases} \quad (5)$$

where MC is the moving contribution and C the contribution. These two parameters are computed by finding the three factors like T_a , T_s , and F_d . Here, the first term refers to the number of test cases that can satisfy availed. The second term refers to the number of test cases that already satisfied c_s and F_d is the decrement factor. The dynamic and adaptive value of the decrement factor is computed using the following equation

$$F_d(t) = T_s^t \quad (6)$$

where T_s^t is the number of test requirement satisfied by the test case t . From this mathematical formulation, we have identified that T_a , T_s , and T_s^t are inversely proportional to the ATAP, but the latter two values control the value of T_a .

3.2 GreedyATAP search algorithm for test case reduction

This paper presents a new algorithm called GreedyATAP for test suite reduction by minimising the test cases after satisfying all the test requirements. The proposed algorithm is the integration of the ATAP measure with the greedy search algorithm. The algorithmic description of the proposed GATAP algorithm is given in Fig. 1.

4 DIV-TBAT: a new optimisation algorithm for test suite reduction

This section presents a new DIV-TBAT algorithm for test suite reduction by injecting the diversity within the TBAT algorithm. In this paper, we have modified the TBAT algorithm by inserting the diversity into the algorithm procedure.

(a) Algorithmic procedure of DIV-TBAT: The algorithmic description of the proposed DIV-TBAT is given in Fig. 2:

Initialisation: We have taken N number of bats in the initial population. The position of every bat is randomly initialised either with 0 or 1. The representation of every bat is given by the following equation

$$B = \{b_{pq}; 0 < p < N; 0 < q < d\} \quad (7)$$

$$b_p = [b_{p1}, b_{p2}, \dots, b_{pd}]; \quad p = 1, 2, \dots, N \quad (8)$$

Here, every bat are represented with d dimensional vector. d is equal to the number of test cases in the test suite. The representation of the every bats is given in Fig. 3. This representation means that the 1, 3, 4, and 6 can be included in the reduced test suite as per the representation. The other variables like loudness A , pulse rate r , iteration I , minimum frequency $Q_{\min}$ and maximum frequency $Q_{\max}$, and velocity of every bat v_i^t are also

$$ATAP(t) = \begin{cases} \infty; & c_s \text{ can be satisfied by } t \text{ only} \\ \frac{\sum_{s=1}^k C(t, c_s) - \sum_{s=1}^l MC(t, c_s)}{g(t)}; & \text{otherwise} \end{cases} \quad (3)$$

1	Algorithm: DIV-TBAT
2	Input: $T \rightarrow$ Test suite
3	$Cost \rightarrow$ Cost vector
4	Output:
5	$x_b \rightarrow$ best solution (Selected test cases)
6	Begin
7	Initialize variables such as, $A, r, I, Q_{\min}, Q_{\max}, \alpha, \beta$
8	Initialize $f=1$, bat population b_p and velocity v_i^f
9	While $f < I$
10	Find fitness for b_p using T and $Cost$
11	Update velocity v_i^f and frequency Q_i
12	Update bats' positions by u_i^f
13	Inject diversity to find x_i^f
14	Find fitness for x_i^f using T and $Cost$
15	Store best solution x_b
16	If ($\gamma > r$)
17	Generate local solution around best solution
18	Endif
19	If ($\gamma < A$)
20	Generate random solution, x_r
21	Find fitness of x_r using T and $Cost$
22	If (fitness(x_r) < fitness(x_b))
23	Update x_b, r and A
24	Endif
25	Endif
26	$f = f + 1$
27	Endwhile
28	Return x_b
29	End

Fig. 2 DIV-TBAT search algorithm for test suite reduction

b_p						
1	0	1	1	0	1	0

Fig. 3 Solution representation of DIV-TBAT algorithm

initialised. The values assigned to these variables are as follows: $A = 0.5$, $r = 0.5$, and $Q_{\min}, Q_{\max} = 1$.

Evaluation: Every bat are evaluated using the fitness function $F(b_p)$. This function aims to minimise the following equation

$$F(b_p) = \begin{cases} \sum_{q=1}^d b_{pq} * g_q & \text{if } S = m \\ \infty & \text{else} \end{cases} \quad (9)$$

$$S = \sum_{j=1}^m z_j \quad (10)$$

$$z_j = \begin{cases} 1: & \text{if } \sum_{k=1}^{d-x} T_R^{kj} > 0 \\ 0: & \text{otherwise} \end{cases} \quad (11)$$

where $d - x$ is equal to the number of ones in b_p (means that number of selected test cases). The fitness function multiplies the position of the bats b_{pq} with the cost vector g_q only if the entire test requirements are satisfied. To check whether all the test requirements are satisfied, the conditional equality of S and m is used. The value of S is obtained by adding the value of all the z . The z is found out by taking the summation of the reduced test suite of the corresponding test requirement.

Movement of virtual bats: Once the fitness function is computed for all the bats, a bat which is having the minimum fitness is assumed to be x_b . To update every position of the bats, velocity is computed based on the following equation defined in TBAT

algorithm. Then, the position of the velocity is updated by adding the position of the old iteration with the new velocity. Then, the position is converted to either zero or one by fixing the threshold value of 0.5

$$Q_i = Q_{\min} + (Q_{\max} - Q_{\min}) * \gamma \quad (12)$$

$$v_i^h = v_i^{h-1} + (\alpha * Q_i(x_i^h - x_b) + \beta(U - x_b * Q_i)) \quad (13)$$

$$y_i^f = x_i^{f-1} + v_i^f \quad (14)$$

$$u_i^f = \begin{cases} 1; & y_i^f < 0.5 \\ 0; & y_i^f \geq 0.5 \end{cases} \quad (15)$$

where γ is a random value which is used to update the frequency of the bat using $Q_{\min}$ and $Q_{\max}$. It ranges between -1 and 1 . Also, U is a unitary vector which is subtracted from the best solution x_b and multiplied with frequency value Q_i . α and β are weighted constants.

Diversity injection using the bitwise operators: The diversity computation is injected into the TBAT algorithm using the proposed bitwise operation. Here, the updated position based on the above equation is taken as u_i^f and the complements of the bit vector, w_i^f , are computed

$$w_i^f = \sim u_i^f \quad (16)$$

Then, the diversity-enabled solution is generated using the following bitwise operation. From the equation, we identify that the first element of the solution vector $x_i^f(1)$ is assumed to be one

$$x_i^f(1) = 1 \quad (17)$$

The even element from 2 to end, $x_i^f(a_o)$, is updated by taking bitwise OR operation between the $u_i^f(a_o)$ and $w_i^f(a_o)$. Similarly, the odd element except the first element will be computed by taking bitwise xor operation in between the $u_i^f(a_e)$ and the $x_i^f(a_e - 1)$. The formulae to compute the solution vector, if is odd number, is given in the following equation. If it is an even number, a_o will be varied until and is varied until to reach d

$$x_i^f(a_o) = u_i^f(a_o) + w_i^f(a_o); \quad a_o = 3, 5, \dots, d \quad (18)$$

$$x_i^f(a_e) = u_i^f(a_e) \oplus x_i^f(a_e - 1); \quad a_e = 2, 4, \dots, d - 1 \quad (19)$$

Here, we include the idea of including diversity between solutions and within the solution. This objective is obtained by including three bitwise operators, such as (i) complement operator to build a diversified solution, (ii) OR operation to select most number of test cases, (iii) XOR operation reject the meaningful number of test cases. These three bitwise operators aim to generate a new position vector, which is diversified enough.

Loudness and pulse rate-based movement: In this step, a random value γ is compared with loudness A and pulse rate r and a new random solution is generated, if the random value is less than these parameters. Otherwise, there is no change in the solution.

Termination: The above steps are executed until the number of iteration is reached to I . Once the number of iteration is equivalent to the I , the best bat indicated as x_b , is taken as the final output and the test suite corresponding to the x_b is the final reduced test suite, T_R .

5 Running example

Table 1 shows the running example of the DIV-TBAT algorithm. Here, we have considered seven test cases and seven test requirement. The test suite selected for running example is given in Table 1.

The input for the DIV-TBAT algorithm is the test pool P and the cost vector C_T . The variables are initialised as, $n = 5$, $t = 2$; $A = 0.5$, $r = 0.5$, $Q_{\min} = 0$; $Q_{\max} = 1$; $d = 7$. Table 1 shows the steps, such as initialisation of x , Q , and v , diversity injection using the proposed formulae, best solution from initialisation, updated value of x , y , Q , and v after first iteration, and its best solution 1. The bat population of x is generated randomly. Then, diversity formulae are injected to

get the new population as shown in Table 1. Then, the fitness for every solution of x is computed based on the developed equation using cost vector C_T . The obtained values of fitness is as: $\text{Fitness}(x(1)) = 90$; $\text{Fitness}(x(2)) = 80$; $\text{Fitness}(x(3)) = 90$; $\text{Fitness}(x(4)) = 40$; $\text{Fitness}(x(5)) = 91$. $x(4)$ is taken as the best solution from initialisation. In the first iteration, for $\gamma = -0.35$, $\alpha = 0.8$, $\beta = 0.2$, frequency, velocity, and position values are updated.

Table 1 Running example of DIV-TBAT algorithm

test pool P and cost vector C_T								
	C_{i1}	C_{i2}	C_{i3}	C_{i4}	C_{i5}	C_{i6}	C_{i7}	Cost (C_T)
C_{1j}	1	1	0	0	0	0	0	1
C_{2j}	0	1	1	0	0	0	0	1
C_{3j}	0	0	1	1	0	0	0	5
C_{4j}	0	0	0	1	1	0	0	11
C_{5j}	0	0	0	0	1	1	0	23
C_{6j}	1	0	0	0	0	1	0	40
C_{7j}	0	0	0	0	0	0	1	10
initialisation of x , Q , and v								
x	0	1	0	0	1	0	1	
	1	0	0	1	0	0	0	
	0	1	0	0	1	0	0	
	1	0	0	1	0	1	0	
	0	0	1	0	1	0	0	
Q	1	1	1	1	1	1	1	
v	0	0	0	0	0	0	0	
	0	0	0	0	0	0	0	
	0	0	0	0	0	0	0	
	0	0	0	0	0	0	0	
	0	0	0	0	0	0	0	
diversity injection using the proposed formulae								
x	1	0	1	1	1	1	1	
	1	1	1	0	1	1	1	
	1	0	1	1	1	1	1	
	1	1	1	0	1	0	1	
	1	1	1	1	1	1	1	
best solution from initialisation								
x_b	1	1	1	0	1	0	1	
updated value of x , y , Q , and v after iteration 1								
v	0.27	0.48	0.27	-0.01	0.27	-0.01	0.27	
	0.27	0.27	0.27	0.2	0.27	-0.01	0.27	
	0.27	0.48	0.27	-0.01	0.27	-0.01	0.27	
	0.27	0.27	0.27	0.2	0.27	0.2	0.27	
	0.27	0.27	0.27	-0.01	0.27	-0.01	0.27	
Q	-0.35	-0.35	-0.35	-0.35	-0.35	-0.35	-0.35	
y	1.27	0.48	1.27	0.99	1.27	0.99	1.27	
	1.27	1.27	1.27	0.2	1.27	0.99	1.27	
	1.27	0.48	1.27	0.99	1.27	0.99	1.27	
	1.27	1.27	1.27	0.2	1.27	0.2	1.27	
	1.27	1.27	1.27	0.99	1.27	0.99	1.27	
x	1	0	1	1	1	1	1	
	1	1	1	0	1	1	1	
	1	0	1	1	1	1	1	
	1	1	1	0	1	0	1	
	1	1	1	1	1	1	1	
after injection	1	1	1	0	1	0	1	
	1	0	1	1	1	0	1	
	1	1	1	0	1	0	1	
	1	0	1	1	1	1	1	
	1	0	1	0	1	0	1	
best solution after iteration 1								
x_b	1	0	1	0	1	0	1	

Then, fitness is computed for every solution of x : $\text{Fitness}(x(1)) = 40$, $\text{Fitness}(x(2)) = 50$; $\text{Fitness}(x(3)) = 40$; $\text{Fitness}(x(4)) = 40$; $\text{Fitness}(x(5)) = 39$. From the first iteration, $x(5)$ is taken as the best solution which indicates that total test cases selected are $S = \{t1, t3, t5, t7\}$ and the total cost is $1 + 5 + 23 + 10 = 39$.

6 Experimental set-up

Here, we discuss the different software programs considered for the study, different evaluation metrics, and existing algorithms taken for comparison study.

6.1 Software programs considered

Software-artefact infrastructure repository (SIR) [23] contains Java, C, C++, and C# programs for experimentation with testing and analysis techniques. From the repository, we have taken eight different subject programs, such as printtoken, printtoken2, replace, schedule, schedule2, space, teas, and totinfo, which were implemented in C programming.

6.2 Research questions

The definition of nine evaluation metrics and research question are given below.

RQ1: Do the reduced test suite show the minimum cost to prove the cost reduction capability?

RQ1 can be answerable only if we find the suite cost of the test suite. It is defined as follows:

$$\text{SuiteCost}(T_R) = \sum_{i=1}^{n-x} g_i \quad (20)$$

where T_R is the reduced test suite, $n - x$ the number of test cases presented in the reduced test suite. g_i is the cost associated with the test case.

RQ2: Do the reduced test suite show the minimum size to prove the size reduction capability?

RQ2 can be answered by finding the size of test suite. The following formula is used to find the suite of the test suite

$$\text{SuiteSize}(T_R) = n - x \quad (21)$$

where n is the number of test cases presented in the original test suite. x is the removed test cases.

RQ3: Can the algorithms able to obtain the test suite with a lesser iterative period?

The iterative period is the steps required to complete the test case selection process. Here, $I(T_R)$ represents the number of iterations required to complete selection process.

RQ4: What is the percentage of suite cost reduction (SCR) when compared with the original test suite?

SCR is computed by taking the relative difference of the cost of the original test suite with the selected test suite

$$\text{SCR}(T_R) = \frac{\text{SuiteCost}(T) - \text{SuiteCost}(T_R)}{\text{SuiteCost}(T)} \quad (22)$$

where $\text{SuiteCost}(T)$ is suite cost of original test suite and $\text{SuiteCost}(T_R)$ the suite cost of reduced test suite.

RQ5: What is the percentage of suite size reduction (SSR) when compared with the original test suite?

SSR is computed by taking the relative difference of the size of the original test suite with the selected test suite

$$\text{SSR}(T_R) = \frac{\text{SuiteSize}(T) - \text{SuiteSize}(T_R)}{\text{SuiteSize}(T)} \quad (23)$$

where $\text{SuiteSize}(T)$ is the suite size of original test suite, and $\text{SuiteSize}(T_R)$ the suite size of reduced test suite.

RQ5: Do the selected test cases produced by the algorithms have many test cases in common?

The common rate (CR) of two algorithms is computed based on the following equation:

$$\text{CR}(T_R^1, T_R^2) = \frac{T_R^1 \cap T_R^2}{T_R^1 \cup T_R^2} \quad (24)$$

where $T_R^1 \cap T_R^2$ is the number of common test cases selected by Algorithms 1 and 2. $T_R^1 \cup T_R^2$ is the number of unique test cases selected by Algorithms 1 and 2.

RQ6: What is the ratio of improvement obtained for the proposed algorithm when compared with other algorithms individually in terms of suite cost?

Average improvement on suite cost for the two algorithms is computed as follows:

$$\begin{aligned} \text{AISC}(T_R^1, T_R^2) \\ = \frac{1}{N_p} \sum_{p=1}^{N_p} \frac{\text{SuiteCost}(T_R^1) - \text{SuiteCost}(T_R^2)}{\text{Max}(\text{SuiteCost}(T_R^1), \text{SuiteCost}(T_R^2))} \end{aligned} \quad (25)$$

where $\text{SuiteCost}(T_R^1)$ is the suite cost associated with Algorithm 1, $\text{SuiteCost}(T_R^2)$ is suite cost associated with Algorithm 2. N_p is the number of programs taken for empirical study.

RQ7: What is the ratio of improvement obtained for the proposed algorithm when compared with other algorithms individually in terms of suite size?

Average improvement on suite size for the two algorithms is computed as follows:

$$\text{AISS}(T_R^1, T_R^2) = \frac{1}{N_p} \sum_{p=1}^{N_p} \frac{\text{SuiteSize}(T_R^1) - \text{SuiteSize}(T_R^2)}{\text{Max}(\text{SuiteSize}(T_R^1), \text{SuiteSize}(T_R^2))} \quad (26)$$

where $\text{SuiteSize}(T_R^1)$ is the suite size associated with Algorithm 1, and $\text{SuiteSize}(T_R^2)$ the suite size associated with Algorithm 2. N_p is the number of programs taken for empirical study.

RQ8: What is the ratio of improvement obtained for the proposed algorithm when compared with other algorithms individually in terms of iterations?

Average improvement on iterations required for the two algorithms is computed as follows:

$$\text{AII}(T_R^1, T_R^2) = \frac{1}{N_p} \sum_{p=1}^{N_p} \frac{I(T_R^1) - I(T_R^2)}{\text{Max}(I(T_R^1), I(T_R^2))} \quad (27)$$

where $I(T_R^1)$ is the number of iterations required for Algorithm 1, and $I(T_R^2)$ the number of iterations required for Algorithm 2. N_p is the number of programs taken for empirical study.

6.3 Experimental steps

The eight software programs considered here for empirical study are written in C language. The implementation of seven algorithms is performed with JAVA using jdk 1.7 and netbeans editor. After collecting the execution times of the tests, we followed an empirical set-up similar to the one described in [24]. Here, we generate 1000 test suites for each program. Table 2 shows the averages of size and execution cost of the 1000 generated test suites for each subject program.

7 Experimental results and discussion

Analysis based on suite cost: Table 3 shows the suite cost of the seven algorithms, such as GreedyRatio (GR), GreedyElIrreplaceability (GE), diversity-based genetic algorithm (DIV-GA), TAP, TBAT, ATAP, and DIV-TBAT, for all the eight programs. GR [24] is a Greedy algorithm developed by integrating

ratio, whereas GE [24] is designed by incorporating ElIrreplaceability into Greedy. ElIrreplaceability is a cost metric formulated by extending Irreplaceability, which is the ratio of the contribution or goodness of a test case to the total cost. Panichella *et al.* [25] introduced a MOGA, coined as DIV-GA, based on the mechanisms of orthogonal design and orthogonal evolution that increase diversity by injecting new orthogonal individuals during the search process for test suite reduction. For printtokens program, the proposed ATAP and DIV-TBAT obtain the value of 43.23 and 40.84. Here, GE and DIV-GA obtain the value of 40.85 and 40.86. Overall, the proposed DIV-GA outperformed three subject programs by showing the minimum cost value.

Analysis based on suite size: Table 4 plots suitesize of seven algorithms for all the eight programs. Here, for printtokens program, the proposed DIV-TBAT and ATAP shows the minimum suite size of 39 which is less than the existing algorithm like GR, GE, and DIV-GA which has obtained the value of 42, 40, and 40, respectively. From this analysis, we justify that the proposed DIV-TBAT shows the better performance for five programs out of eight programs considered for the experimental study.

Analysis based on SCR and SSR: Table 5 shows the SCR and SSR of seven algorithms for all the eight programs. From the table, we understand that the DIV-TBAT shows the better SCR value of 0.9698 in printtokens program. For this software program, the existing GR, GE, and DIV-GA obtain the value of 0.9676, 0.9698, and 0.9698, respectively. Overall, the proposed DIV-TBAT shows the high performance in three subject programs and also, shows the second level performance in other programs considered for experimental study. From the table, the existing GR, GE, and DIV-GA algorithms obtain the SSR value of 0.965, 0.9667, and 0.9667 for printtokens program. The proposed DIV-TBAT obtains the higher value of 0.9675. The proposed DIV-TBAT and ATAP

outperformed in all the eight programs by showing the maximum value in terms of SSR.

8 Comparative study

Analysis based on common rate: Fig. 4 shows the common rate of seven algorithms for the four programs such as printtoken, printtoken2, replace, and schedule. Fig. 4a shows the common rate of the algorithms in the case of printtokens program. The proposed DIV-TBAT shows the common rate in the range of 0.9 which indicates that the DIV-TBAT identifies the more common test cases as like the other algorithms. Fig. 4b shows the common rate of the algorithms in the case of printtokens2 program. Here, the proposed ATAP select the unique test case by showing the lesser value when compared with other algorithms. Fig. 4c shows the common rate of seven algorithms in the case of replace program. The proposed ATAP and DIV-TBAT select the most common test cases because it shows the values in the range of 0.99. Fig. 4d shows the common rate of the algorithms in the case of schedule program. Here, the common rate is very distributed.

Fig. 5 shows the common rate of seven algorithms for the four programs such as Schedule2, space, tcas, and totinfo. Fig. 5a shows the common rate matrix for schedule 2 program. Here, ATAP shows the less value when compared with other algorithms. Fig. 5b shows the common rate of the seven algorithms in the case of space program. Here, common rate is distributed in all the elements. Fig. 5c shows the common rate of the seven algorithms for tcas program. Fig. 5d shows the common rate of the algorithms in the case of totinfo program.

Analysis based on AISC, AISS, and AII: Table 6 shows the AISC, AISS, and AII of seven algorithms for all the eight programs. When performing the comparison of all the algorithms

Table 2 Descriptions of the SIR subject programs and test suites

Programs	Number of test requirements	Size of test pool	Test suite size	Average execution cost of test case, ms	Average execution cost of test suite, ms
Printtoken	132	4130	1100	1.1259	1238.49
Printtoken2	99	4115	1100	0.8472	931.92
replace	97	5542	2600	0.8246	2143.96
schedule	41	2650	1700	0.3611	613.87
schedule2	56	2710	1600	0.5287	845.92
space	937	13,585	6300	18.8457	118,727.91
tcas	11	1608	1100	0.1209	132.99
totinfo	59	1052	800	0.6036	482.88

Table 3 Suite cost of seven algorithms for all the eight programs

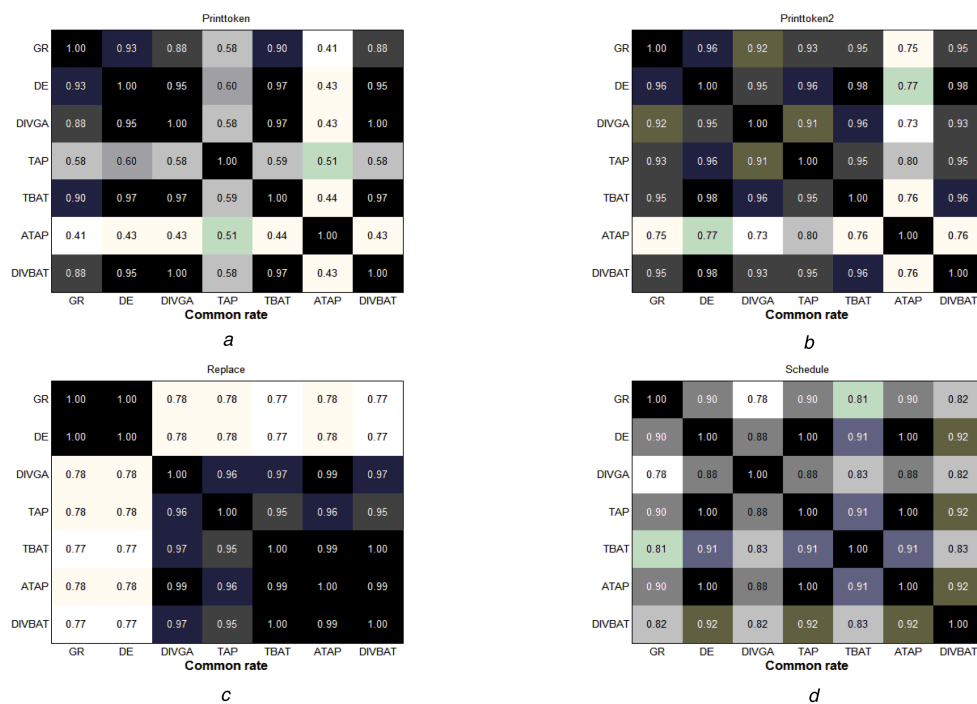
	Suite cost							
	Printtokens	Printtokens 2	Replace	Schedule	Schedule2	tcas	Totinfo	Space
GR	43.72	48.21	109.36	38.16	43.52	9.23	29.36	111.16
GE	40.85	46.61	110.29	39.64	43.59	10.07	29.48	111.16
DIV-GA	40.86	47.73	113.87	36.18	42.13	9.17	26.74	113.25
TAP	69.43	46.73	113.87	36.18	42.08	10.11	28.34	118.72
TBAT	40.84	45.96	114.23	39.64	43.64	9.26	26.46	111.16
ATAP	43.23	47.58	115.86	39.64	44.52	10.28	28.65	118.72
DIV-TBAT	40.84	45.85	112.37	38.14	42.94	9.27	26.18	113.25

Table 4 Suitesize of seven algorithms for all the eight programs

	SuiteSize							
	Printtokens	Printtokens 2	Replace	Schedule	Schedule2	tcas	Totinfo	Space
GR	42	58	136	91	85	62	51	7
GE	40	56	137	91	85	62	51	7
DIV-GA	40	57	137	80	83	60	50	7
TAP	64	56	137	91	85	62	51	7
TBAT	39	55	137	90	83	60	50	7
ATAP	39	56	139	91	83	62	52	7
DIV-TBAT	39	56	135	85	83	60	50	7

Table 5 SCR and SSR of seven algorithms for all the eight programs

	SCR							
	Printtokens	Printtokens 2	Replace	Schedule	Schedule2	tcas	Totinfo	Space
GR	0.9676	0.9483	0.949	0.9368	0.9485	0.9292	0.9396	0.999
GE	0.9698	0.95	0.9485	0.9343	0.9485	0.9227	0.9394	0.999
DIV-GA	0.9698	0.9488	0.9468	0.94	0.9502	0.9297	0.945	0.9989
TAP	0.9486	0.9499	0.9468	0.94	0.9502	0.9224	0.9417	0.9989
TBAT	0.9698	0.9507	0.9467	0.9343	0.9484	0.929	0.9456	0.999
ATAP	0.968	0.949	0.9459	0.9343	0.9474	0.9211	0.9411	0.9989
DIV-TBAT	0.9698	0.9508	0.9475	0.9368	0.9492	0.9289	0.9462	0.9989
SSR								
GR	0.965	0.9473	0.9477	0.9465	0.9469	0.9436	0.9363	0.9989
GE	0.9667	0.9491	0.9473	0.9465	0.9469	0.9436	0.9363	0.9989
DIV-GA	0.9667	0.9482	0.9473	0.95	0.9481	0.9455	0.9375	0.9989
TAP	0.9467	0.9491	0.9473	0.9465	0.9469	0.9436	0.9363	0.9989
TBAT	0.9675	0.95	0.9473	0.9471	0.9481	0.9455	0.9375	0.9989
ATAP	0.9675	0.9491	0.9465	0.9465	0.9481	0.9436	0.935	0.9989
DIV-TBAT	0.9675	0.9491	0.9481	0.95	0.9481	0.9455	0.9375	0.9989

**Fig. 4** Common rate of seven algorithms for the four programs such as Printtoken, Printtoken2, replace, and schedule (a) Printtoken, (b) Printtoken2, (c) Replace, (d) Schedule

versus GR, the proposed DIV-TBAT shows the good performance by showing the higher AISC value of 0.0234. The proposed DIV-TBAT shows the AISS value of 0.0226 when doing the comparison of GE algorithm versus other algorithms. For the comparison of all the algorithms with GR, the DIV-TBAT outperforms by obtaining the AII value of 0.0709 which is higher than the existing algorithms.

9 Statistical analysis

Pairwise comparisons are the simplest kind of statistical tests that a researcher can apply within the framework of an experimental study. Table 7 shows the w -test on a data set for different combination of algorithms for the values of α are 0.01 and 0.1. For the value of α is 0.01, the p -value attained by the proposed DIV-TBAT with respect to GR is 0.0274, 0.0845, 0.0655, 0.0345, 0.0210, 0.0358, and 0.0154 and the p -value attained by the DIV-TBAT with respect to DIV-GA is 0.0145, 0.0358, 0.0125, 0.0741, 0.0129, 0.0465, and 0.0742. From Table 7, we can identify that the statistical test almost always returns lower p -values for the proposed DIV-TBAT than for other existing algorithms and more

often rejects the null hypothesis. Overall, it is known that the proposed DIV-TBAT model is more likely to reject the null hypothesis.

10 Threats to validity

Search-based software engineering (SBSE) experiments share the limitations born out of the immaturity in both its source areas. One of these limitations regards the lack of a list of validity threats that may affect SBSE experiments. The threats can be categorised into four major classes: conclusion, internal, construct, and external threats.

Conclusion validity threats: One of the important conclusion validity threats is about the random variation of the initial solution. In order to handle the random initialisation issues, the proposed DIV-TBAT is experimented with 1000 times for various random data and the average is taken as final metric value. The second important aspect is the lack of good descriptive measure. The proposed DIV-TBAT algorithm solved this solvency by comparing with baseline algorithms based on different descriptive measures.

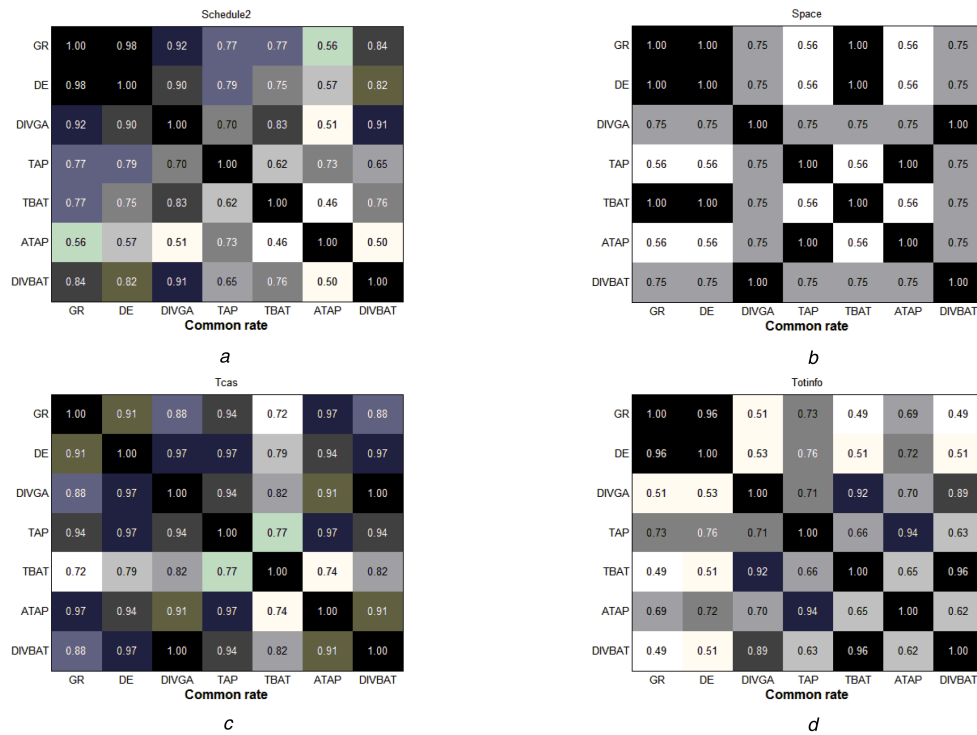


Fig. 5 Common rate of seven algorithms for the four programs such as Schedule2, space, tcas, and totinfo
(a) Schedule2, (b) Space, (c) tcas, (d) totinfo

Table 6 AISC, AISS, and All of seven algorithms for all the eight programs

	AISC						
	GR	GE	DIV-GA	TAP	TBAT	ATAP	DIV-TBAT
GR	0	0.0045	-0.0246	0.0513	-0.0157	0.0292	-0.0234
GE	-0.0045	0	-0.0287	0.0441	-0.0202	0.025	-0.028
DIV-GA	0.0234	0.028	0	0.0722	0.008	0.0532	-0.0009
TAP	-0.0513	-0.0441	-0.0731	0	-0.0645	-0.0216	-0.0722
TBAT	0.0157	0.0202	-0.0085	0.0645	0	0.0453	-0.008
ATAP	-0.0292	-0.025	-0.0538	0.0216	-0.0453	0	-0.0532
DIV-TBAT	0.0246	0.0287	0.0009	0.0731	0.0085	0.0538	0
	AISS						
GR	0	-0.0094	-0.0317	0.0396	-0.0253	-0.0111	-0.0318
GE	0.0094	0	-0.0223	0.0469	-0.0162	-0.0019	-0.0226
DIV-GA	0.0317	0.0223	0	0.0692	0.0064	0.0204	0.0002
TAP	-0.0396	-0.0469	-0.0692	0	-0.0619	-0.0476	-0.0683
TBAT	0.0253	0.0162	-0.0064	0.0619	0	0.0142	-0.0065
ATAP	0.0111	0.0019	-0.0204	0.0476	-0.0142	0	-0.0207
DIV-TBAT	0.0318	0.0226	-0.0002	0.0683	0.0065	0.0207	0
	All						
GR	0	-0.0093	-0.0402	0.0396	-0.0453	-0.011	-0.0709
GE	0.0093	0	-0.0311	0.0469	-0.0364	-0.0019	-0.0627
DIV-GA	0.0402	0.0311	0	0.078	-0.0054	0.0292	-0.0336
TAP	-0.0396	-0.0469	-0.078	0	-0.0821	-0.0476	-0.1084
TBAT	0.0453	0.0364	0.0054	0.0821	0	0.0345	-0.0285
ATAP	0.011	0.0019	-0.0292	0.0476	-0.0345	0	-0.0607
DIV-TBAT	0.0709	0.0627	0.0336	0.1084	0.0285	0.0607	0

Internal validity threats: This threat deals with two important aspects, such as poor parameter settings and lack of clear data. In the proposed method, the parameters are explicitly given in the paper to obtain the better results. Also, the parameters of the baseline methods are fixed according to the parametric values given in the corresponding methods. Also, this paper experiments with the standard benchmark data for doing the experimentation. This avoids the problem of lacking of clear of data collection tools and procedures.

Construct validity threats: These threats are concerned with the relations between theory and observation, ensuring that the treatment reflects the construct of the cause and that the outcome reflects the construct of the effect. The important validity threat here is the lack of assessing the validity of cost measures. The proposed method brings the cost metrics as fitness evaluation which is widely accepted for software testing.

External threat: These threats are concerned with the generalisation of observed results to a larger population, outside the sample instances used in the experiment. Major external threats to

Table 7 W-test on p -value ($\alpha = 0.01$) and p -value ($\alpha = 0.1$)

	DIV-TBAT versus GR [26]	DIV-TBAT versus GE [24]	DIV-TBAT versus DIV-GA [21]	DIV-TBAT versus TAP [21]	DIV-TBAT versus TBAT [22]	DIV-TBAT versus ATAP	DIV-TBAT versus GR [26]	DIV-TBAT versus GE [24]	DIV-TBAT versus DIV-GA [21]	DIV-TBAT versus TAP [21]	DIV-TBAT versus TBAT [22]	DIV-TBAT versus ATAP
	$\alpha = 0.01$						$\alpha = 0.1$					
printtokens	0.0274	0.0285	0.0145	0.0294	0.0475	0.0652	0.0008	0.0052	0.0158	0.0257	0.0428	0.0624
printtokens2	0.0845	0.0823	0.0358	0.0687	0.0324	0.0584	0.0500	0.0524	0.0398	0.0425	0.0261	0.0721
replace	0.0655	0.0524	0.0125	0.0695	0.0651	0.0452	0.0845	0.0431	0.0547	0.0724	0.0354	0.0429
schedule	0.0345	0.0254	0.0741	0.0425	0.0327	0.0761	0.0155	0.0356	0.0259	0.0519	0.0329	0.0426
schedule2	0.0210	0.0354	0.0129	0.0321	0.0841	0.0347	0.0155	0.0186	0.0357	0.0458	0.0243	0.0156
tcas	0.0358	0.0574	0.0465	0.0657	0.0245	0.0291	0.0254	0.0554	0.0348	0.0469	0.0547	0.0461
space	0.0154	0.0286	0.0742	0.0673	0.0223	0.0358	0.0598	0.0358	0.0468	0.0258	0.0325	0.0258

SBSE experiments include the varying size and complexity. Therefore, an SBSE approach must be evaluated across a breadth of problem instances, both varying in size and complexity. To provide an assessment on the limits of the proposed DIV-TBAT algorithm, eight data sets are utilised with various size of test requirement and test suite size.

11 Conclusion and future scope

This paper proposed two methods for test suite reduction based on the ATAP measure and DIV-TBAT algorithm. Specifically, in the first method, ATAP measure is newly developed to find the reduced test suite with the help of greedy search algorithm. In the second method, we proposed DIV-TBAT, a novel diversity injected BAT algorithm which combines the popular BAT algorithm with the diversity-preserving mechanism formulated in this paper for test suite reduction. These two proposed methods are validated extensively based on the nine research questions. An empirical study is conducted on eight software programs from SIR repository and evaluation is performed with five existing algorithms using nine evaluation metrics. From the result, we proved that the reduced test suite shows that DIV-TBAT outperforms the entire existing methods in test suite reduction problem. In future, we plan to verify the test suite reduction approaches for fault detection capability.

12 References

- [1] Hemmati, H., Arcuri, A., Briand, L.: 'Achieving scalable model-based testing through test case diversity', *ACM Trans. Softw. Eng. Methodol. (TOSEM)*, 2013, **22**, (1)
- [2] Shaccour, E., Zaraket, F., Masri, W.: 'Coverage specification for test case intent preservation in regression suites'. Proc. of IEEE Sixth Int. Conf. on Software Testing, Verification and Validation Workshops, Luxembourg, 2013
- [3] Nagar, R., Kumar, A., Kumar, S., *et al.*: 'Implementing test case selection and reduction techniques using meta-heuristics'. Proc. of 5th Int. Conf. on Confluence the Next Generation Information Technology Summit (Confluence), Noida, India, 2014, pp. 837–842
- [4] Yabin Wang, Y., Ruizhi Gao, R., Zhenyu Chen, Z., *et al.*: 'WAS: a weighted attribute-based strategy for cluster test selection', *J. Syst. Softw.*, 2014, **98**, pp. 44–58
- [5] Harrold, M.J., Gupta, R., Soffa, M.L.: 'A methodology for controlling the size of a test suite', *ACM Trans. Softw. Eng. Methodol.*, 1993, **2**, (3), pp. 270–285
- [6] Jeffrey, D., Gupta, N.: 'Improving fault detection capability by selectively retaining test cases during test suite reduction', *IEEE Trans. Softw. Eng.*, 2007, **33**, (2), pp. 108–123
- [7] Coutinho, A.E.V.B., Cartaxo, E.G., Machado, P.D.D.L.: 'Analysis of distance functions for similarity-based test suite reduction in the context of model-based testing', *Softw. Qual. J.*, 2016, **24**, (2), pp. 407–445
- [8] Dandan, G., Tiantian, W., Xiaohong, S., *et al.*: 'A test-suite reduction approach to improving fault-localization effectiveness', *Comput. Lang., Syst. Struct.*, 2013, **39**, (3), pp. 95–108
- [9] Lin, C.T., Tang, K.W., Chen, C.D., *et al.*: 'Reducing the cost of regression testing by identifying irreplaceable test cases'. Proc. of the 6th Int. Conf. on Genetic and Evolutionary Computing, Kitakushu, Japan, August 2012, pp. 257–260
- [10] Bryce, R.C., Colbourn, C.J., Cohen, M.B.: 'A framework of greedy methods for constructing interaction test suites'. Proc. of Int. Conf. on Software Engineering, Saint Louis, MO, USA, 2005, pp. 146–155
- [11] Cohen, M., Dwyer, M., Shi, J.: 'Constructing interaction test suites for highly-configurable systems in the presence of constraints: a greedy approach', *IEEE Trans. Softw. Eng.*, 2008, **34**, (5), pp. 633–650
- [12] Parsa, S., Khalilian, A.: 'A bi-objective model inspired greedy algorithm for test suite minimization', *Future Gener. Inf. Technol., Lect. Notes Comput. Sci.*, 2009, **5899**, pp. 208–215
- [13] Tallam, S., Gupta, N.: 'A concept analysis inspired greedy algorithm for test suite minimization'. Proc. of the 6th ACM SIGPLAN-SIGSOFT Workshop on Program Analysis for Software Tools and Engineering, New York, NY, USA, 2006, **31**, (1), pp. 35–42
- [14] Xu, S., Miao, H., Gao, H.: 'Test suite reduction using weighted set covering techniques'. Proc. of 13th ACIS Int. Conf. on Software Engineering, Artificial Intelligence, Networking and Parallel & Distributed Computing (SNPD), Kyoto, Japan, 2012, pp. 307–312
- [15] Zhong, H., Zhang, L., Mei, M.: 'An experimental study of four typical test suite reduction techniques', *Inf. Softw. Technol.*, 2008, **50**, (6), pp. 534–546
- [16] Chen, T.Y., Lau, M.F.: 'A simulation study on some heuristics for test suite reduction', *Inf. Softw. Technol.*, 1998, **40**, (13), pp. 777–787
- [17] Chen, T.Y., Lau, M.F.: 'A new heuristic for test suite reduction', *Inf. Softw. Technol.*, 1998, **40**, (5–6), pp. 347–354
- [18] Ma, X.Y., He, Z.F., Sheng, B.K., *et al.*: 'A genetic algorithm for test-suite reduction'. Proc. of the IEEE Int. Conf. on Systems, Man and Cybernetics, Waikoloa, HI, USA, October 2005, pp. 133–139
- [19] Lucia, A.D., Penta, M.D., Oliveto, R., *et al.*: 'On the role of diversity measures for multi-objective test case selection'. Proc. of 7th Int. Workshop Automation of Software Test, Zurich, Switzerland, 2012, pp. 145–151
- [20] Harris, P., Raju, N.: 'A greedy approach for coverage-based test suite reduction', *Int. Arab J. Inf. Technol.*, 2015, **12**, (1), pp. 17–23
- [21] Sugave, S.R., Patil, S.H., Reddy, B.E.: 'A cost-aware test case minimization approach using TAP measure and greedy search algorithm', *J. Intell. Syst., Commun.*, 2017, **10**, (4), pp. 60–69
- [22] Sugave, S.R., Patil, S.H., Reddy, B.E.: 'Cost-aware test suite minimization approach using TBAT optimization algorithm for software testing', *Int. J. Intell. Eng. Syst.*, 2017, **10**, (2), pp. 21–43
- [23] Software-artifact infrastructure repository (SIR). Available at <http://sir.unl.edu/content/sir.php>
- [24] Lin, C.T., Tang, K.W., Kapfhammer, G.M.: 'Test suite reduction methods that decrease regression testing costs by identifying irreplaceable tests', *Inf. Softw. Technol.*, 2014, **56**, (10), pp. 1322–1344
- [25] Panichella, A., Oliveto, R., Di Penta, M., *et al.*: 'Improving multi-objective test case selection by injecting diversity in genetic algorithms', *IEEE Trans. Softw. Eng.*, 2015, **41**, (4), pp. 358–383
- [26] Smith, A.M., Kapfhammer, G.M.: 'An empirical study of incorporating cost into test suite reduction and prioritization'. Proc. of the 24th ACM Symp. on Applied Computing, Software Engineering Track, ACM, New York, NY, USA, March 2009, pp. 461–467